กระปุกออมสิน (piggyBank)

โปรแกรมตรวจมีกระปุกออมสินอยู่ใบเดียว เริ่มต้นมีเงิน 0 บาทเมื่อเริ่มรันโปรแกรม แล้วเรียกฟังก์ชัน `piggyBank(op, x)` ซ้ำหลายครั้งเพื่อทำรายการกับกระปุกใบนี้ (ตั้งแต่ครั้งที่ 1 ไปจนจบการรัน ยอดเงินในกระปุกต่อเนื่องกัน)

op	ความหมาย	สิ่งที่ฟังก์ชันต้องทำ
1	ฝากเงิน x บาท	เพิ่มยอดอีก x แล้วคืนยอดใหม่
2	ถอนเงิน x บาท	ถ้า $x \le$ ยอดปัจจุบัน: ลดยอดลง x แล้วคืนยอดใหม่ ถ้า x มากกว่ายอด: ไม่เปลี่ยนยอดและคืน -1
3	ดูยอดเงิน (x เป็น 0 เสมอ)	คืนยอดปัจจุบัน ไม่เปลี่ยนยอด

ต่างจากฟังก์ชันอื่นที่แต่ละครั้งเรียกเป็นอิสระต่อกัน ข้อนี้ผลลัพธ์ขึ้นกับรายการก่อนหน้า ฟังก์ชันจึงต้องจำยอดเงินไว้ในตัวแปรที่ไม่หายไปเมื่อฟังก์ชันจบ (ตัวแปร global หรือตัวแปร `static` ในฟังก์ชัน)

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันต่อไปนี้ตามที่ประกาศในไฟล์ `piggy_bank.h` โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน

```
long long piggyBank(int op, int x);
```

- `op`: ชนิดรายการ (1 ฝาก, 2 ถอน, 3 ดูยอด)
- `x`: จำนวนเงิน (เป็น 0 เมื่อ op $= 3$)
- ค่าที่คืนกลับ: ยอดเงินในกระปุกหลังทำรายการ (ชนิด `long long`) หรือ -1 เมื่อถอนไม่ได้เพราะยอดไม่พอ

การส่งงาน: ดาวน์โหลดไฟล์แนบ `piggy_bank.cpp` (ไฟล์โครง) และ `piggy_bank.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `piggy_bank.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `piggy_bank.cpp` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจตัวอย่างสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp piggy_bank.cpp -o piggy_bank`)

ข้อสำคัญ: โปรแกรมตรวจเรียกฟังก์ชันนี้ Q ครั้งต่อกรณีทดสอบตามลำดับในข้อมูลนำเข้า ห้ามแก้ไขไฟล์ `piggy_bank.h` และชื่อกับชนิดของพารามิเตอร์ต้องตรงกับที่ประกาศไว้ทุกตัวอักษร แต่ละกรณีทดสอบเริ่มในโปรเซสใหม่ ตัวแปรธรรมดา (local) ภายในฟังก์ชันจะถูกสร้างใหม่ทุกครั้งที่เรียกและหายไปเมื่อฟังก์ชันจบ ส่วนตัวแปร `static` ภายในฟังก์ชันและตัวแปร global อยู่ตลอดการรันโปรแกรม ระวังช่วงค่าของชนิดข้อมูล: ยอดสะสมอาจถึง 10^{14}

ตัวอย่างการเรียกฟังก์ชัน

เหตุการณ์	ค่าที่คืน	ยอดเงินในกระปุกหลังเรียก
`piggyBank(1, 100)`	คืน 100	100
`piggyBank(3, 0)`	คืน 100	100
`piggyBank(2, 30)`	คืน 70	70
`piggyBank(2, 100)`	คืน -1	70
`piggyBank(3, 0)`	คืน 70	70

ตารางแสดงการเรียกต่อเนื่องกันตามลำดับในการรันเดียว ยอดเงินเริ่มต้นเป็น 0

ข้อจำกัด

- $2 \le Q \le 100\ 000$
- op เป็น 1, 2 หรือ 3
- $1 \le x \le 1\ 000\ 000\ 000$ เมื่อ op เป็น 1 หรือ 2 และ $x = 0$ เมื่อ op $= 3$
- เวลา 1 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) op เป็น 1 หรือ 3 เท่านั้น และ $x \le 1\ 000$
2. (30 คะแนน) op เป็น 1, 2 หรือ 3, $x \le 1\ 000$ และไม่มีการถอนที่ถูกปฏิเสธ
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

โปรแกรมตรวจตัวอย่างอ่านข้อมูลจากอินพุตมาตรฐาน บรรทัดแรกเป็นจำนวนเต็มบวก Q คือจำนวนครั้งที่เรียกฟังก์ชัน จากนั้นเป็นข้อมูลของการเรียกแต่ละครั้งตามลำดับ รวม Q ครั้ง ดังนี้
แต่ละบรรทัดมีจำนวนเต็ม op และ x คั่นด้วยช่องว่าง (การเรียกฟังก์ชันตามลำดับ)

ค่าที่ฟังก์ชันคืนกลับสำหรับแต่ละครั้งที่เรียก บรรทัดละหนึ่งค่า (ผลลัพธ์ที่ต่อไปนี้ใช้ฟังก์ชันที่ทำงานถูกต้อง)

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
5 1 100 3 0 2 30 2 100 3 0	100 100 70 -1 70

ฝาก 100 $\rightarrow$ 100 · ดูยอด $\rightarrow$ 100 · ถอน 30 $\rightarrow$ 70 · ถอน 100 ไม่พอ $\rightarrow -1$ (ยอดยังเป็น 70) · ดูยอด $\rightarrow$ 70
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
3 3 0 1 5 3 0	0 5 5

กระปุกว่างอยู่ → 0 · ฝาก 5 → 5 · ดูยอด → 5
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3